

第3讲 Android开发基础

一、课程基本信息

项目	内容
课程名称	移动应用开发
授课章节	第二章 原生开发基础（第3讲）
授课学时	2学时
授课方式	理论讲授 + 案例演示
授课教师	刘东良
适用专业	软件工程

二、教学目标

知识目标

- 掌握Kotlin语言基础：变量、函数、类、空安全
- 理解Activity生命周期各阶段及调用顺序
- 熟悉常用UI控件：TextView、Button、EditText、RecyclerView
- 掌握Logcat调试工具的使用方法

能力目标

- 能够使用Kotlin编写基本的Android程序
- 能够正确管理Activity生命周期并处理状态保存
- 能够使用常用UI控件构建简单界面
- 能够使用Logcat进行调试和问题定位

素质目标

- 培养严谨的工程思维与问题溯源能力

2. 树立规范编码、注重细节的开发意识
3. 增强自主学习新技术的能力与信心

三、教学重点与难点

教学重点

1. Kotlin变量、函数与类的基本用法
2. Activity生命周期及各回调方法的作用
3. 常用UI控件的属性与事件处理
4. Logcat调试工具的使用

教学难点

1. Kotlin空安全机制的理解与应用
2. Activity生命周期状态切换的实际场景分析
3. RecyclerView的适配器模式与复用机制

四、教学内容与过程设计

第一学时：Kotlin语言基础与Activity生命周期

1. 课程导入（8分钟）

- **复习导入**：回顾上一讲移动应用开发技术体系，原生开发的地位与价值
- **提问引导**：Android应用是用什么语言开发的？一个页面从创建到销毁经历了什么？
- **学习目标**：明确本讲重点——掌握Kotlin基础、理解Activity生命周期、学会使用UI控件

2. Kotlin语言基础（22分钟）

变量声明

- `val`：只读变量（不可变），对应Java的final
- `var`：可变变量
- 类型推断：Kotlin可自动推断变量类型
- 示例代码：

```
val name: String = "Android"
var age: Int = 10
val pi = 3.14 // 类型推断为Double
```

函数定义

- 函数声明语法: `fun 函数名(参数): 返回类型 { ... }`
- 单表达式函数: 可省略花括号和return
- 默认参数与命名参数
- 示例代码:

```
fun add(a: Int, b: Int): Int {
    return a + b
}

fun greet(name: String, greeting: String = "Hello") {
    println("$greeting, $name!")
}
```

类与对象

- 类声明与构造函数
- 属性与成员方法
- 数据类 (data class) 的便捷性
- 示例代码:

```
class User(val name: String, var age: Int) {
    fun introduce() {
        println("我是$name, 今年${age}岁")
    }
}

data class Student(val id: String, val name: String, val score: Int)
```

空安全

- 可空类型与非空类型: `String?` vs `String`
- 安全调用: `?.`
- 非空断言: `!!`
- Elvis操作符: `?:`
- 空安全的意义: 从语言层面避免空指针异常
- 示例代码:

```
var name: String? = null
println(name?.length) // 安全调用，返回null
println(name ?: "未知") // Elvis操作符，提供默认值
```

3. Activity生命周期 (15分钟)

生命周期概述

- Activity是Android应用的核心组件
- 生命周期：从创建到销毁的完整过程
- 7个核心回调方法：onCreate → onStart → onResume → onPause → onStop → onDestroy
- 附加方法：onRestart

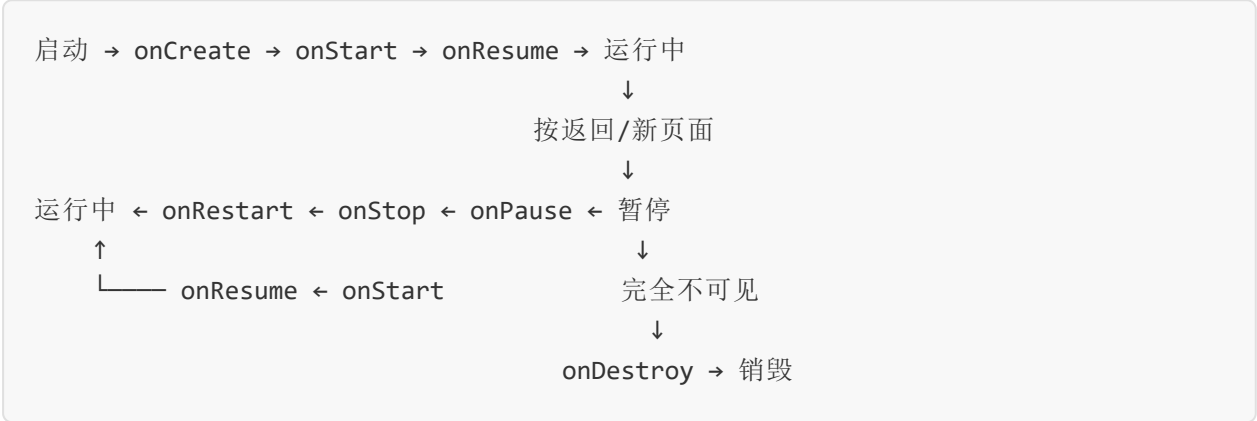
各阶段详解

- `onCreate()` : Activity首次创建时调用，初始化界面、数据绑定，仅调用一次
- `onStart()` : Activity变为可见时调用
- `onResume()` : Activity获取焦点、可交互时调用
- `onPause()` : Activity失去焦点、不可交互时调用，保存轻量数据
- `onStop()` : Activity完全不可见时调用
- `onDestroy()` : Activity销毁前调用，释放资源

典型场景分析

- 场景1: 启动新Activity → onPause → onStop
- 场景2: 返回原Activity → onRestart → onStart → onResume
- 场景3: 屏幕旋转 → onDestroy → onCreate (重建)
- 场景4: 按Home键 → onPause → onStop

生命周期示意图



第二学时：UI控件与Logcat调试

4. 常用UI控件（22分钟）

TextView（文本视图）

- 作用：显示文本内容
- 常用属性：text、textSize、textColor、textStyle、gravity
- 示例：

```
<TextView
    android:id="@+id/tvTitle"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="欢迎使用"
    android:textSize="24sp"
    android:textColor="#000000" />
```

Button（按钮）

- 作用：响应用户点击操作
- 常用属性：text、background、enabled
- 点击事件处理：setOnClickListener
- 示例：

```
btnSubmit.setOnClickListener {
    // 处理点击逻辑
    Toast.makeText(this, "点击了按钮", Toast.LENGTH_SHORT).show()
}
```

EditText（输入框）

- 作用：接收用户文本输入
- 常用属性：hint、inputType、maxLines、password
- 获取输入内容：editText.text.toString()
- 示例：

```
<EditText
    android:id="@+id/etUsername"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="请输入用户名"
    android:inputType="text" />
```

RecyclerView (列表视图)

- 作用：展示大量数据列表，高效复用视图
- 核心组件：Adapter（适配器）、ViewHolder（视图持有者）、LayoutManager（布局管理器）
- 工作原理：回收复用屏幕外的ItemView，提升性能
- 基本使用步骤：
 1. 添加依赖
 2. 定义Item布局
 3. 创建Adapter和ViewHolder
 4. 设置LayoutManager和Adapter

5. Logcat调试工具 (13分钟)

Logcat简介

- Android Studio内置的日志查看工具
- 显示系统及应用的日志信息
- 支持按级别、标签、进程过滤

日志级别

- `Log.v()`：Verbose（详细），最低级别
- `Log.d()`：Debug（调试），开发调试用
- `Log.i()`：Info（信息），一般信息
- `Log.w()`：Warn（警告），潜在问题
- `Log.e()`：Error（错误），错误异常
- 级别越高，信息越重要

使用示例

```
private const val TAG = "MainActivity"

Log.d(TAG, "onCreate: Activity创建了")
Log.i(TAG, "用户点击了登录按钮")
Log.e(TAG, "网络请求失败", exception)
```

调试技巧

- 合理使用TAG，便于过滤
- 关键节点打印日志，追踪流程
- 异常时打印堆栈信息，定位问题
- 使用过滤器：按日志级别、关键字、包名过滤

6. 案例演示：登录页面实现（10分钟）

- 界面组成：两个EditText（用户名、密码）、一个Button（登录）、一个TextView（标题）
- 功能实现：点击按钮获取输入内容，验证非空，弹出提示
- 演示在Android Studio中编写布局和逻辑代码
- 演示使用Logcat调试输入内容获取是否正确

五、学前测验

测验说明

本测验旨在检测学习者对Android开发基础知识的了解程度，帮助确定学习起点。

测验题目

第1题：在Kotlin中，声明一个不可变变量应使用哪个关键字？

- A. var
- B. val
- C. const
- D. final

正确答案： B

答案解析： Kotlin中使用 `val` 声明只读（不可变）变量，对应Java的final变量。 `var` 用于声明可变量。 Kotlin中没有 `const` 和 `final` 作为变量修饰关键字（const用于顶层常量和伴生对象）。

第2题： Android Activity生命周期中，哪个方法在Activity首次创建时调用，且仅调用一次？

- A. onStart()
- B. onResume()
- C. onCreate()
- D. onRestart()

正确答案： C

答案解析： `onCreate()` 是Activity创建时第一个被调用的方法，用于完成初始化工作（如加载布局、绑定数据），且在Activity的整个生命周期中只调用一次。 onStart和onResume在生命周期中可能被多次调用。

第3题：以下哪个UI控件用于接收用户的文本输入？

- A. TextView
- B. Button
- C. EditText
- D. ImageView

正确答案： C

答案解析： EditText是Android中专门用于接收用户文本输入的控件。TextView用于显示文本，Button用于按钮点击，ImageView用于显示图片。

第4题： Kotlin的空安全机制中，安全调用操作符是哪个？

- A. !!
- B. ?.
- C. ?:
- D. ==

正确答案： B

答案解析： `?.` 是安全调用操作符，当对象不为空时调用方法/访问属性，为空时直接返回null。`!!` 是非空断言（可能抛出空指针），`?:` 是Elvis操作符（提供默认值），`==` 是相等比较。

第5题： 在Android Logcat中，哪个日志级别用于输出错误异常信息？

- A. Log.d()
- B. Log.i()
- C. Log.w()
- D. Log.e()

正确答案： D

答案解析： Log.e()用于输出Error级别的错误信息，通常用于记录异常和严重错误。Log.d()是调试级别，Log.i()是信息级别，Log.w()是警告级别。

测验结果评估

- **5题全对：** 优秀，您对Android开发基础有很好的认知
- **3-4题正确：** 良好，具备基本概念，需要深化理解
- **1-2题正确：** 及格，需要系统学习Android开发基础
- **0题正确：** 需努力，建议从入门知识开始学习

六、课后作业与预习

课后作业

1. 使用Kotlin编写一个简单的计算器函数，实现加减乘除四则运算
2. 简述Activity生命周期的7个方法及各自的作用，并举例说明什么场景下会触发onPause
3. 使用TextView、EditText、Button实现一个简单的用户注册页面（包含用户名、密码、确认密码、注册按钮），点击按钮时使用Toast提示注册结果
4. 在代码中合理添加Log.d日志，使用Logcat观察Activity生命周期各方法的调用顺序

下节预告

- iOS开发概述：ViewController架构、SwiftUI基础组件
- Xcode调试工具（理论讲授与演示为主）
- 原生与跨平台对比：原生开发的性能优势与开发成本分析
- 课程思政：从系统底层逻辑培养严谨的工程思维

实验预告

- 实验二：原生应用开发
- 内容：Android方向使用Kotlin实现登录页面，iOS方向使用SwiftUI实现登录页面
- 要求：实现输入验证、Activity跳转与数据传递

七、教学反思

(课后填写)